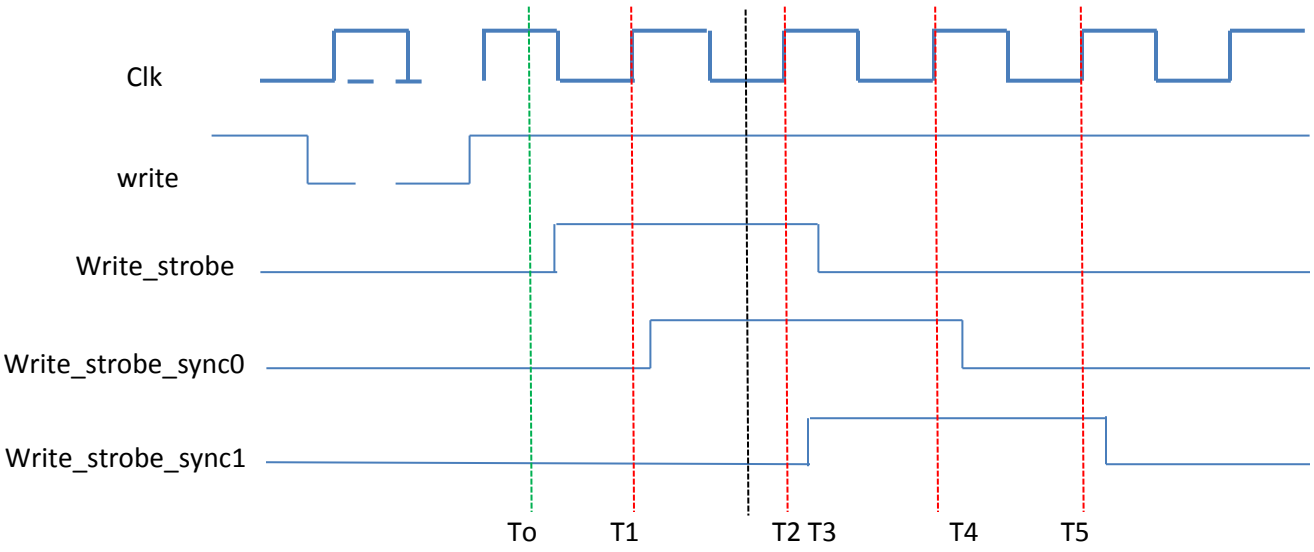
Useful VHDL Codes

I. Pulse generation from a push on a <u>debounced</u> switch

```
latch_write: PROCESS (write, write_strobe_sync1, resetn)
BEGIN
-- asynchronous clear when write_strobe_sync1 goes high
    IF(write_strobe_sync1 = '1' OR resetn = '0') THEN
        write_strobe <= '0';                        --clear write strobe
    ELSIF(write'EVENT AND write = '1') THEN
        write_strobe <= '1';                        -- set write strobe when write goes high
    END IF;                                         -- cleared when write_strobe_sync1 goes high
END PROCESS;

sync_write: PROCESS (clk, resetn)
BEGIN
    IF(resetn = '0') THEN
        write_strobe_sync0 <= '0';
        write_strobe_sync1 <= '0';
    ELSIF(clk'EVENT AND clk = '1') THEN
        write_strobe_sync0 <= write_strobe;         -- synchronizer
        write_strobe_sync1 <= write_strobe_sync0;
    END IF;
END PROCESS;
```



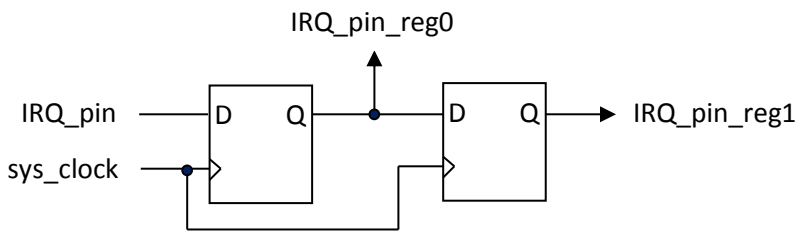
To : Write signal positive edge latches write_strobe to high

T1, T2, T4, T5 : Write_strobe signal is sampled on positive edge of clk to get write_strobe_sync0

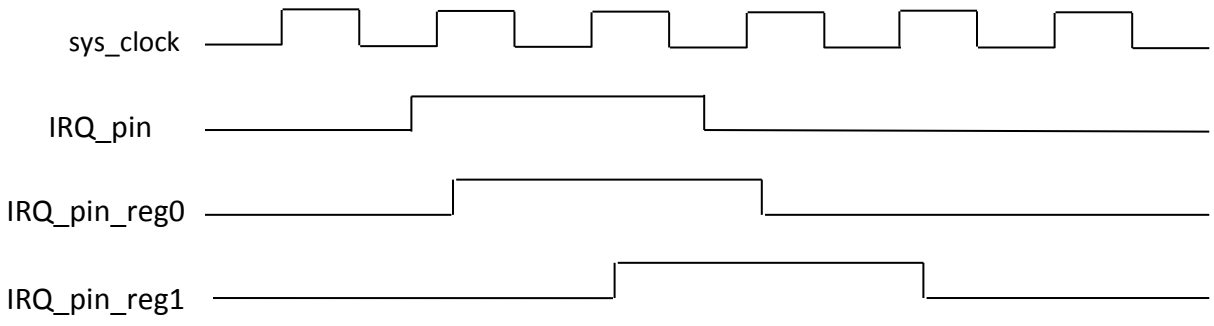
T1, T2, T4, T5 : Write_strobe_sync0 is sampled on positive edge of clk to get write_strobe_sync1

T3 : Write_strobe_sync1 going high clears write_strobe

II. Signal Edge Detection and Interrupt



IRQ_pin is signal on a pin .
IRQ_pin_reg0 is IRQ_pin sampled at T.
IRQ_pin_reg1 is IRQ_pin sampled at T+ Δ (Δ = one clock period)



1. **Positive Edge Detection**
   detect_positive_edge <= IRQ_pin_reg0 AND NOT(IRQ_pin_reg1); -- active high

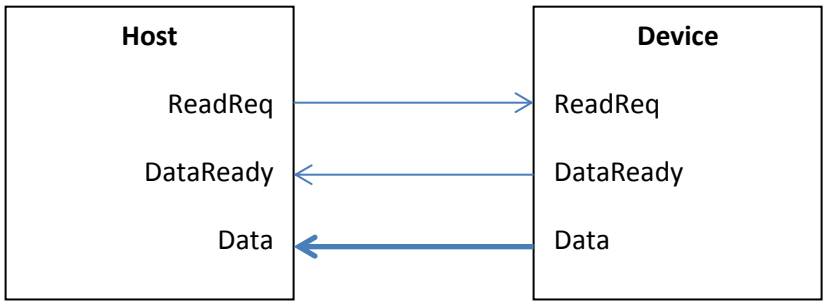
2. **Negative Edge Detection**
   detect_negative_edge <= NOT(IRQ_pin_reg0) AND IRQ_pin_reg1; -- active high

3. **Either Edge Detection**
   detect_either_edge <= IRQ_pin_reg0 XOR IRQ_pin_reg1; -- active high

```
detect_positive_edge <=  IRQ_pin_reg0  AND NOT(IRQ_pin_reg1);      -- active high
detect_negative_edge <= NOT(IRQ_pin_reg0)  AND  IRQ_pin_reg1;      -- active high
detect_either_edge <= IRQ_pin_reg0   XOR  IRQ_pin_reg1;            -- active high
```

```
PROCESS(clk)
BEGIN
  IF (rising_edge(clk)) then
        IF(resetn = '0') then
              irq_pin_reg0 <= '0';
              irq_pin_reg1 <= '0';
        else
              irq_pin_reg0 <= irq_pin;              --implement IRQ_pin_reg0
              irq_pin_reg1 <= irq_pin_reg0;         --implement IRQ_pin_reg1
        END IF;
  END IF;
END PROCESS;
```

III. Handshake between a device and a host.

Before the host reads the device data, the host sends a read-request ReadReq, and then waits for data ready signal DataReady to be active. When DataReady is active, device data is ready to be picked up.

```
Handshake: PROCESS( ReadReq, ReadyFlag)          -- code inside device
BEGIN
   IF ( ReadReq = '1') THEN --ReadReq is set by the host
        DataReady <= '0'; -- not ready
   ELSIF (ReadyFlag'EVENT AND ReadyFlag = '1') THEN
        DataReady <= '1'; -- ready. ReadyFlag is set by the device when data is ready
   END IF;
END PROCESS;
```